

Task 1 - MapReduce Programming 1: Basics

1.1 Number of Occurrences of Lottery Numbers

Note: We assume that the meganumber is a regular lottery number and include occurrences as meganumbers in the count of occurrences of lottery numbers.

1	117
2	152
3	139
4	134
5	131
6	124
7	141
8	127
9	144
10	140
11	135
12	138
13	130
14	138
15	137
16	111
17	124
18	115
19	112
20	135
21	123
22	116
23	105
24	124
25	133
26	104
27	117
28	109
29	125
30	109
31	127
32	99
33	104
34	106
35	136
36	135
37	97
38	129
39	126
40	118

41	108
42	120
43	103
44	117
45	122
46	124
47	81
48	110
49	88
50	106
51	111
52	97
53	102
54	81
55	74
56	89
57	14
58	19
59	15
60	12
61	13
62	16
63	15
64	14
65	12
66	15
67	10
68	19
69	13
70	13
71	13
72	13
73	13
74	17
75	9

1.2 Descending Order by Number of Occurrences of Lottery Numbers

2	152
9	144
7	141
10	140
3	139
12	138
14	138
15	137
35	136
11	135
20	135
36	135
4	134
25	133
5	131
13	130
38	129
8	127
31	127
39	126
29	125
6	124
17	124
24	124
46	124
21	123
45	122
42	120
40	118
1	117
27	117
44	117
22	116
18	115
19	112
16	111
51	111
48	110
28	109
30	109
41	108
34	106
50	106

23	105
26	104
33	104
43	103
53	102
32	99
37	97
52	97
56	89
49	88
47	81
54	81
55	74
58	19
68	19
74	17
62	16
59	15
63	15
66	15
57	14
64	14
61	13
69	13
70	13
71	13
72	13
73	13
60	12
65	12
67	10
75	9

Task 2 - MapReduce Programming 2: Joins

2.1 Explanation of Generic Reduce-Side Join

Step	Explanation
Map	Read records from all datasets, extract join key, emit (joinKey, taggedRecord) where the tag identifies the source dataset.
Combine	Optional local aggregation. Usually not used for joins because all records must remain available.
Partition	Assign records to reducers based on the join key so that all matching keys go to the same reducer.
Shuffle	Transfer mapper outputs across the network and group records with the same join key.
Sort	Sort and group records by join key before reduction.
Reduce	Separate records by source dataset and perform the join. Generate matching output records for 1:1, 1:n, and n:m relationships.

2.2 Taxi Aggregate

dropoff_date	Distance	Earnings
2015-06-01	137223.43	1409103.74
2015-06-02	131085.87	1366296.25
2015-06-03	139624.75	1378500.47
2015-06-04	152275.73	1472577.35
2015-06-05	180112.00	1741694.44
2015-06-06	220171.50	2005130.62
2015-06-07	208587.16	1795657.66
2015-06-08	3212.61	16144.99

2.3 Taxi Join

Date	Gross Margin (Buffalo)
2015-06-01	1391258.25
2015-06-02	1349495.10
2015-06-03	1359883.84
2015-06-04	1452988.83
2015-06-05	1718017.75
2015-06-06	1976291.25
2015-06-07	1768727.39
2015-06-08	

There is no entry for 2015-06-08 because a standard reduce-side join acts as an inner join. It requires the date to exist in both datasets simultaneously (`dropoff_date` from the taxi dataset and `businessday` from the fuel dataset) to calculate a result. If either of the two datasets lacks records for June 8, 2015 (e.g., no taxi trips were recorded or no fuel price entry was logged for that specific date), the join condition fails, and the day is completely omitted from the final output.

Task 3 - Hive

3.1

The following HiveQL query was used to determine the five most frequently drawn individual lottery numbers.

```
SELECT CAST(num AS int) AS number, COUNT(*) AS occurrences
FROM (
    -- Get the space-separated numbers
    SELECT explode(split(lotterynumbers, ' ')) AS num FROM lottery
    UNION ALL
    -- Combine them with the extra number column as string
    SELECT CAST(meganumber AS STRING) AS num FROM lottery
) t
GROUP BY CAST(num AS INT) --cast to int to avoid "02" and "2" differences
ORDER BY occurrences DESC
LIMIT 5;
```

The query splits the `lotterynumbers` column into individual numbers, creates one row per number using `explode`, counts the occurrences of each number, sorts the result in descending order and returns the top five numbers.

Number	Occurrences
2	152
9	144
7	141
10	140
3	139

3.2

To estimate the theoretical gross margin of taxi operations, the aggregated taxi data was joined with the fuel dataset. Following the assumption from Task 2.3, a fuel consumption of 21.3 miles per gallon was used and the Buffalo fuel price was applied.

The following HiveQL query was used:

```
SELECT
    t.dropoff_date,
    ((t.total_earnings -
      ((f.buffalo_price * t.total_distance) / 21.3)))
    AS gross_margin_buffalo
FROM (
    SELECT
        dropoff_date,
        SUM(distance) AS total_distance,
        SUM(earnings) AS total_earnings
    FROM taxi
    GROUP BY dropoff_date
) t
JOIN fuel f
ON t.dropoff_date = f.businessday
ORDER BY t.dropoff_date;
```

The query first aggregates the taxi data by day and then joins it with the fuel dataset using the corresponding date. The theoretical gross margin is calculated as the total earnings minus the estimated fuel costs based on the Buffalo fuel price and a fuel efficiency of 21.3 miles per gallon.

Date	Gross Margin (Buffalo)
2015-06-01	1391258.25
2015-06-02	1349495.10
2015-06-03	1359883.84
2015-06-04	1452988.83
2015-06-05	1718017.75
2015-06-06	1976291.25
2015-06-07	1768727.39

The results show the theoretical gross margin for each day after accounting for estimated fuel costs. This demonstrates how Hive can be used to combine multiple datasets and perform analytical calculations on large-scale data.